MATA KULIAH • MACHINE LEARNING

Decision Tree

CART • ID3 • Pruning

Pertemuan 5

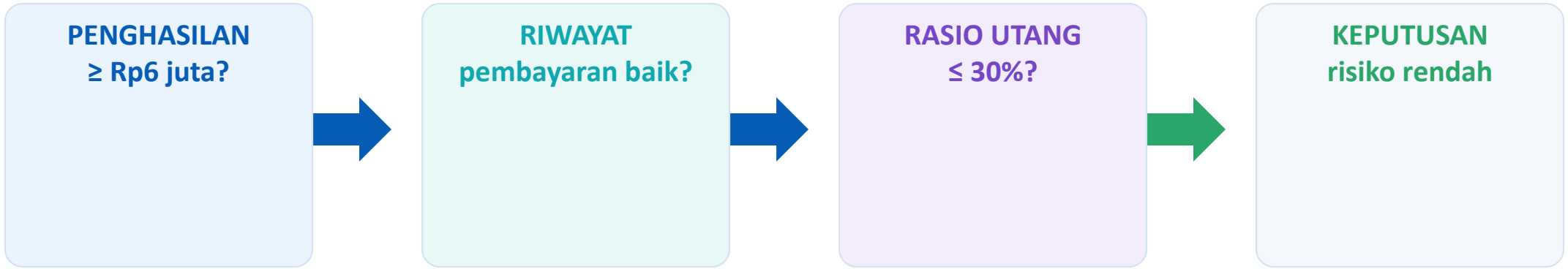
Membaca logika model, menghitung kualitas split, dan mengendalikan kompleksitas pohon



Program Studi Teknik Informatika • FTIK

Satu keputusan dapat diurai menjadi rangkaian pertanyaan

Contoh: persetujuan kredit sederhana

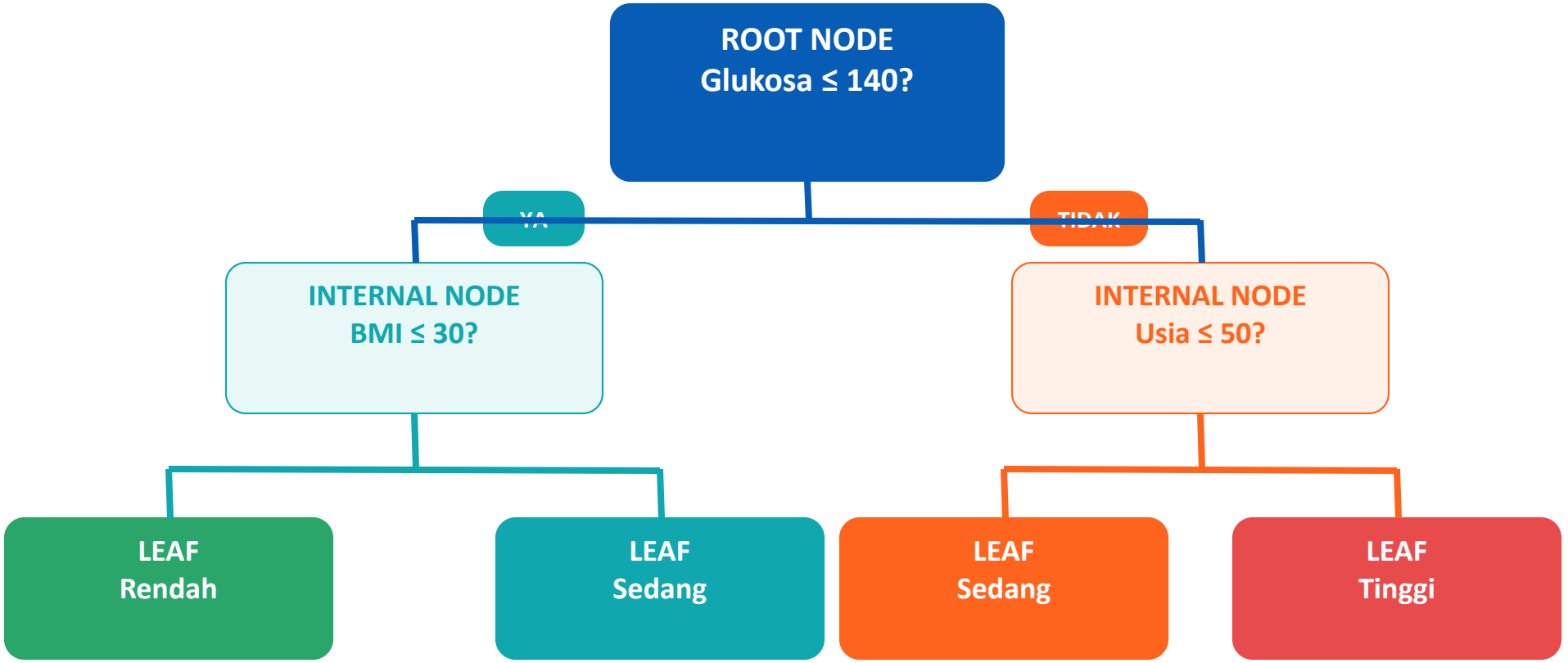


Decision Tree belajar pertanyaan dan urutannya dari data latih.

Setelah pertemuan ini, Anda mampu…

01	**Menjelaskan struktur pohon**	root, internal node, branch, dan leaf
02	**Menghitung kualitas split**	Gini, Entropy, dan Information Gain
03	**Membandingkan CART dan ID3**	kriteria, jenis cabang, dan jenis tugas
04	**Melatih serta memangkas pohon**	pre-pruning, post-pruning, dan evaluasi

Struktur pohon menunjukkan dari mana keputusan berasal



Satu prediksi mengikuti tepat satu jalur dari root menuju leaf.

Pohon dibangun dengan membelah data secara rekursif

1. CARI
uji fitur dan threshold

2. PILIH
penurunan impurity terbesar

3. BELAH
data kiri dan kanan

4. ULANGI
hingga kriteria berhenti

STOP jika: node murni • max_depth tercapai • sampel terlalu sedikit

Leaf menghasilkan kelas atau nilai, sesuai jenis tugas

KLASIFIKASI

Target berupa kategori.

Leaf memprediksi kelas mayoritas atau probabilitas kelas.

Contoh: spam / bukan spam

REGRESI

Target berupa angka kontinu.

Leaf memprediksi rata-rata target pada data yang mencapai leaf.

Contoh: harga rumah

Scikit-learn memakai keluarga CART untuk DecisionTreeClassifier dan DecisionTreeRegressor.

Gini mengukur peluang salah jika label dipilih acak

$$Gini(t) = 1 - \Sigma_k \, p_k^2$$

MURNI

10 positif • 0 negatif

Gini = 0

CAMPURAN

6 positif • 4 negatif

Gini = 0,48

SEIMBANG

5 positif • 5 negatif

Gini = 0,50

Semakin kecil Gini, semakin homogen isi node.

Contoh: node 6 positif dan 4 negatif memiliki Gini 0,48

$p_+ = 6/10 = 0{,}6 \qquad p_- = 4/10 = 0{,}4$

$$\text{Gini} = 1 - (0{,}6^2 + 0{,}4^2)$$

1 Kuadratkan proporsi

$0{,}6^2 = 0{,}36$
$0{,}4^2 = 0{,}16$

2 Jumlahkan

$0{,}36 + 0{,}16 = 0{,}52$

3 Kurangi dari 1

$1 - 0{,}52 = 0{,}48$

Nilai 0,48 berarti node masih cukup bercampur.

Entropy mengukur tingkat ketidakpastian dalam node

$$\text{Entropy}(t) = -\Sigma_k \, p_k \log_2(p_k)$$

Bayangkan mengambil satu data tanpa melihat labelnya.

0

Pasti

node murni

≈ 0,5

Mulai tidak pasti

kelas dominan

1

Paling tidak pasti

dua kelas seimbang

Node 60%–40% memiliki Entropy sekitar 0,971

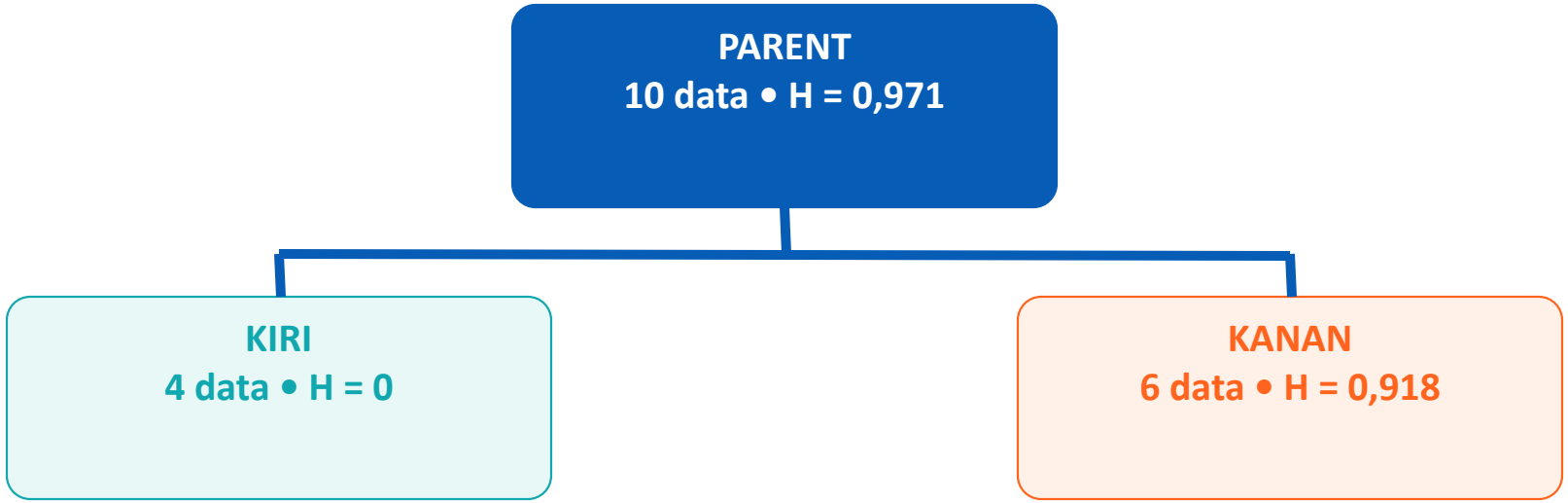
$$H = -[0{,}6 \log_2(0{,}6) + 0{,}4 \log_2(0{,}4)]$$

$\log_2(0{,}6)$	$\approx -0{,}737$	$0{,}6 \times (-0{,}737) = -0{,}442$
$\log_2(0{,}4)$	$\approx -1{,}322$	$0{,}4 \times (-1{,}322) = -0{,}529$

$$H = -(-0{,}442 - 0{,}529) = 0{,}971$$

Information Gain memilih split dengan penurunan terbesar

$$IG = H(\text{parent}) - \Sigma_j \left(n_j / n\right) H(\text{child}_j)$$



Split terbaik memiliki Information Gain paling besar.

Split contoh mengurangi Entropy sebesar 0,420

H tertimbang = (4/10 × 0) + (6/10 × 0,918)

BOBOT KIRI

4/10 × 0 = 0

BOBOT KANAN

6/10 × 0,918 = 0,551

TOTAL

0 + 0,551 = 0,551

IG = 0,971 – 0,551 = 0,420

Bandingkan nilai 0,420 dengan seluruh kandidat fitur dan threshold.

Gini dan Entropy biasanya memilih split yang serupa

Aspek	Gini	Entropy
Rumus	$1 - \Sigma p^2$	$-\Sigma p \log_2 p$
Rentang 2 kelas	0 sampai 0,5	0 sampai 1
Komputasi	Lebih sederhana	Memakai logaritma
Umum dipakai	CART default	ID3 / criterion='entropy'

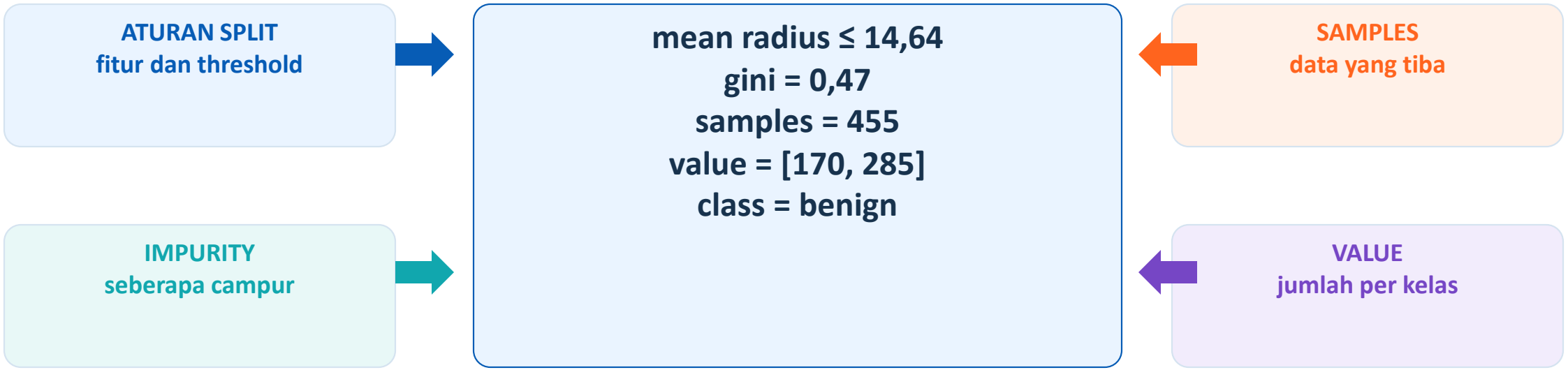
Pilih berdasarkan validasi; jangan menganggap satu kriteria selalu unggul.

CART dan ID3 berbeda pada kriteria, cabang, dan dukungan tugas

Aspek	CART	ID3
Kriteria split	Gini; dapat Entropy	Information Gain
Cabang	Selalu biner	Dapat multi-cabang
Tugas	Klasifikasi + regresi	Umumnya klasifikasi
Fitur numerik	Threshold langsung	Perlu diskritisasi
Scikit-learn	Tersedia langsung	Tidak tersedia langsung

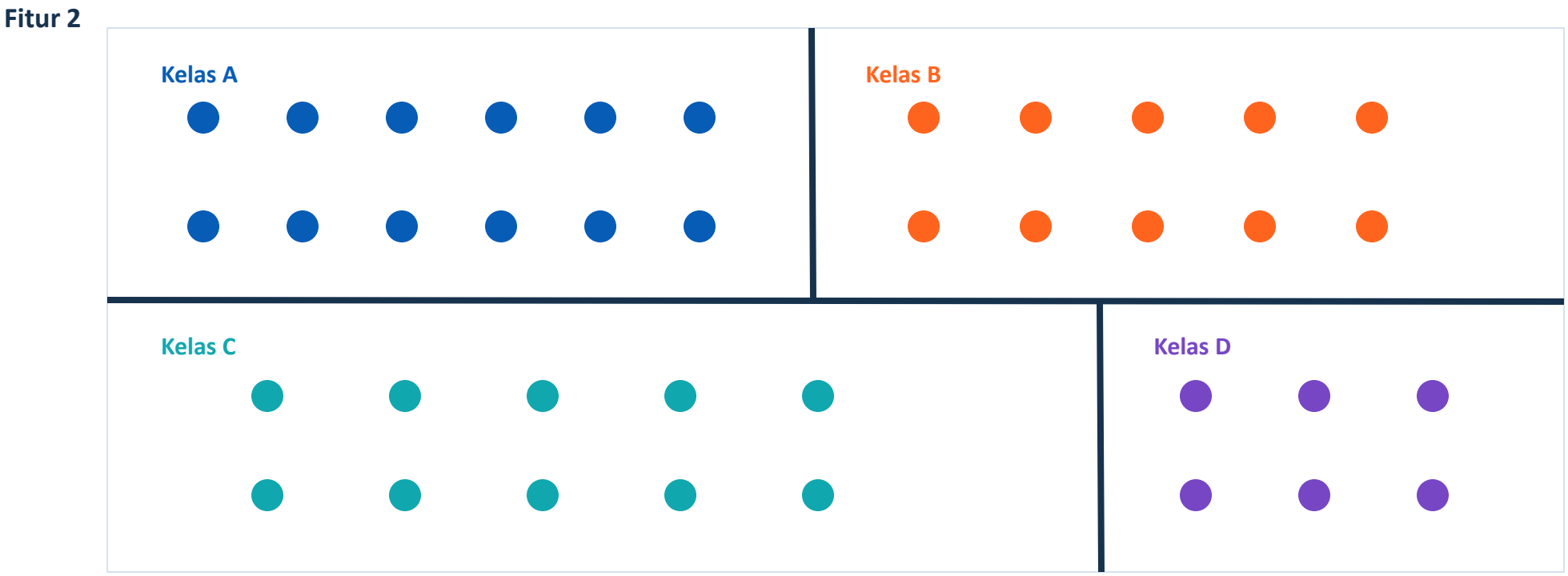
DecisionTreeClassifier di scikit-learn menggunakan keluarga CART.

Satu node menyimpan aturan, impurity, sampel, dan kelas



CLASS adalah kelas mayoritas; warna semakin pekat berarti node semakin murni.

Split pohon membentuk batas keputusan yang sejajar sumbu



Kelebihan: aturan mudah dibaca. Keterbatasan: batas miring memerlukan banyak split.

Pohon yang terlalu dalam dapat menghafal noise



POHON PENUH

PRUNING

POHON RINGKAS

Train sangat tinggi + test jauh lebih rendah = indikasi overfitting.

Pre-pruning menghentikan pohon sebelum terlalu rumit

max_depth

Batas kedalaman maksimum

Mulai uji: 2, 4, 6

min_samples_split

Minimum data untuk membelah node

Nilai besar → split lebih sedikit

min_samples_leaf

Minimum data di setiap leaf

Mencegah leaf sangat kecil

Nilai terbaik dipilih dengan cross-validation, bukan dari akurasi train.

Post-pruning menyeimbangkan kesalahan dan kompleksitas

$$C\alpha(T) = R(T) + \alpha\,|T|$$

POHON A

R(T) = 0,18
|T| = 8 leaf
α = 0,02

Cα = 0,18 + 0,02(8) = 0,34

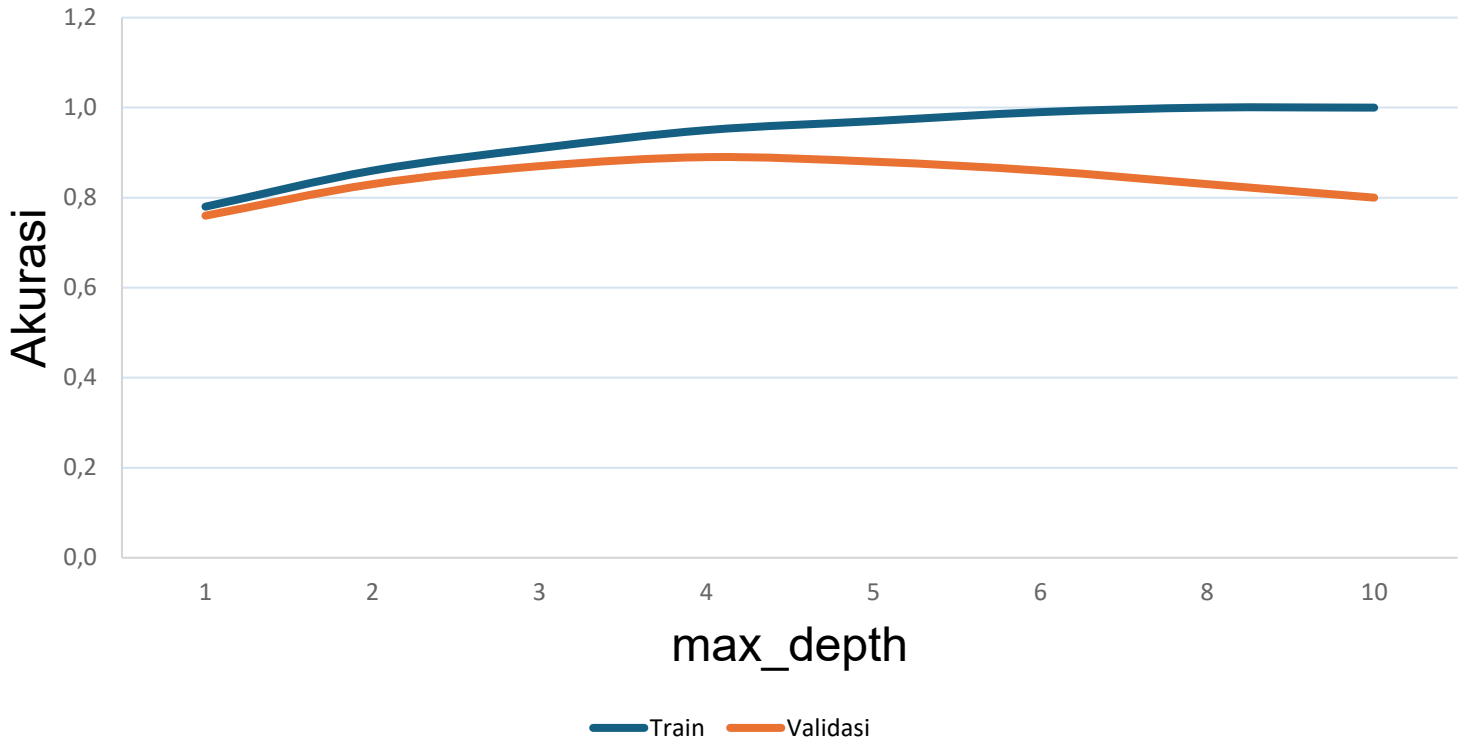
POHON B

R(T) = 0,22
|T| = 4 leaf
α = 0,02

Cα = 0,22 + 0,02(4) = 0,30

Dengan α = 0,02, Pohon B dipilih: sedikit lebih salah, tetapi jauh lebih sederhana.

Validation curve memilih kedalaman tanpa mengintip test



BACA KURVA

Train terus naik.

Validasi mencapai puncak sekitar depth = 4.

Setelah itu gap melebar: model mulai overfitting.

Test dipakai satu kali setelah hyperparameter dipilih dari train/CV.

Decision Tree unggul dalam interpretasi, tetapi mudah tidak stabil

KELEBIHAN

- aturan mudah dijelaskan
- tidak perlu scaling
- menangkap hubungan non-linear
- mendukung klasifikasi dan regresi
- menyediakan feature importance

KETERBATASAN

- rentan overfitting
- perubahan kecil data dapat mengubah pohon
- batas split sejajar sumbu
- sering kalah akurat dari ensemble
- importance dapat menyesatkan

Interpretabilitas tetap harus disertai validasi dan pemeriksaan bias.

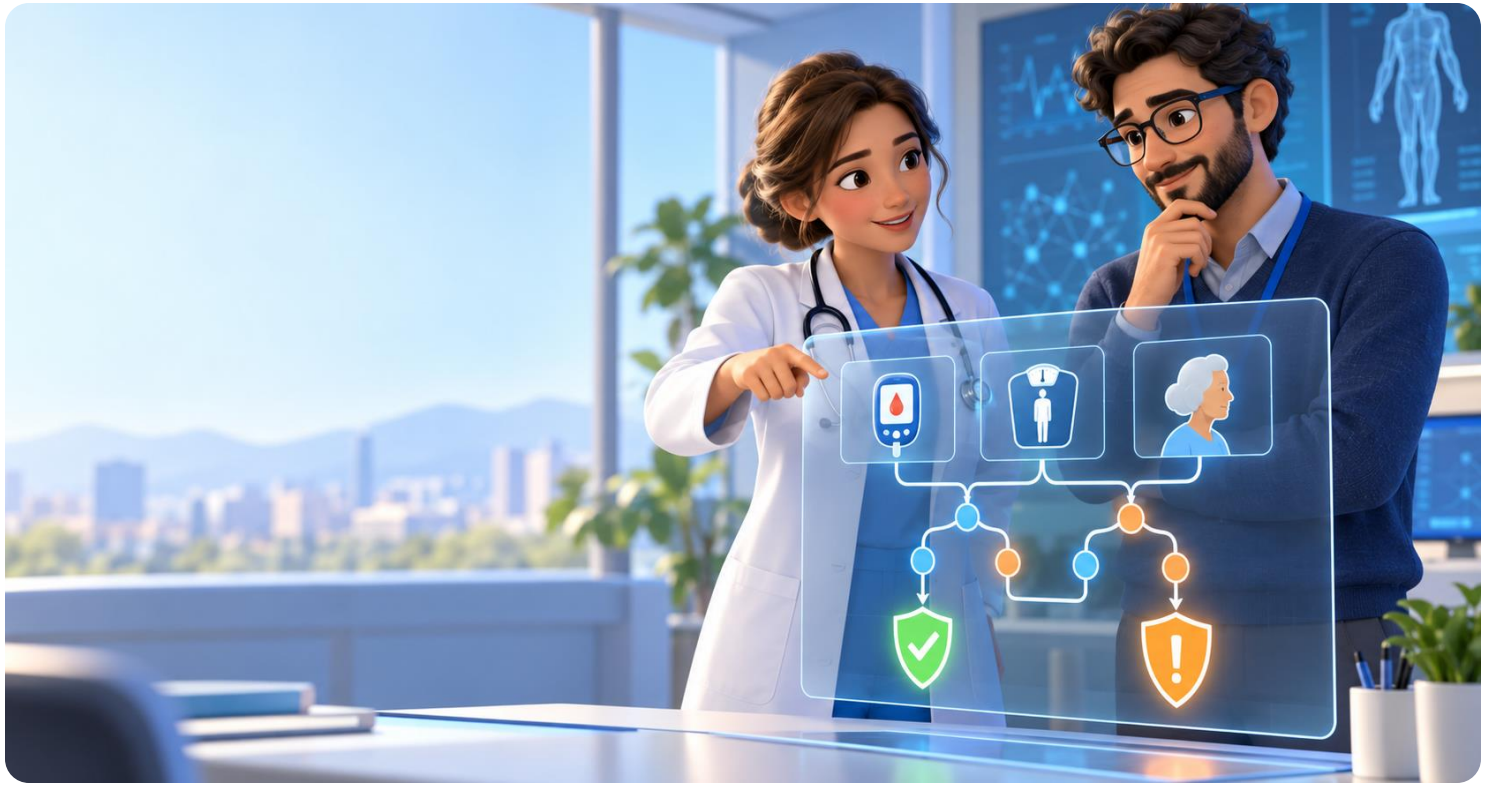
Pruning memperbaiki generalisasi pada studi kasus diabetes

Studi kasus modul

Pohon penuh
Train 99% • Test 71%
→ menghafal data latih

Setelah pruning
Train 85% • Test 79%
→ gap menyempit

Kedalaman menjadi 4–5 level.



Model membantu interpretasi risiko; keputusan klinis tetap memerlukan tenaga medis.

Kode membandingkan pohon penuh dan pohon yang dibatasi

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

tree_full = DecisionTreeClassifier(random_state=42)
tree_pruned = DecisionTreeClassifier(
    max_depth=4,
    min_samples_leaf=5,
    random_state=42
)

for name, model in [
    ("Full", tree_full),
    ("Pruned", tree_pruned)
]:
    model.fit(X_train, y_train)
    tr = accuracy_score(y_train, model.predict(X_train))
    te = accuracy_score(y_test, model.predict(X_test))
    print(name, tr, te, "gap =", tr - te)
```

BANDINGKAN

Akurasi train
Akurasi test
Gap train – test

INTERPRETASI

Gap besar → overfitting
Keduanya rendah → underfitting

Praktikum: pilih pohon yang sederhana tetapi tetap akurat

1. Muat breast cancer dataset
2. Split stratified 80:20
3. Uji max_depth: 2, 4, 6, None
4. Uji min_samples_leaf: 1, 5, 10
5. **Bandingkan train, CV, test, leaf**

OUTPUT WAJIB

- tabel parameter
- akurasi train dan CV
- akurasi test final
- gap train–test
- jumlah leaf node
- visualisasi pohon terpilih

Pilih hyperparameter dari CV, bukan dari skor test berulang.

Pertanyaan analisis: mengapa pohon dengan train tertinggi belum tentu terbaik?

Kesimpulan

1. Prediksi adalah satu jalur pertanyaan dari root menuju leaf.
2. Split dipilih karena paling besar mengurangi impurity.
3. Gini dan Entropy mengukur campuran kelas dari sudut berbeda.
4. Pruning menukar sedikit kecocokan train untuk generalisasi.
5. Hyperparameter dipilih dari CV; test digunakan untuk evaluasi akhir.

Jika parent H = 0,90 dan entropy tertimbang child = 0,55, berapa Information Gain-nya?